

Sometimes this can confuse you, but it is very powerful. If you combine SSH with the *tmux* terminal multiplexer, then you have even more possibilities. With the *-t* switch, SSH can do a lot of remote operations resourcefully.

Key-based authentication

Now, let's find out how to set up SSH public key authentication to log in without passwords. This involves several steps. First, run these on host A, from which you want to log in to host B:

```
$ ssh-keygen -t dsa
<Keep empty private key password>
$ cat ~/.ssh/id_dsa.pub | ssh hostB "cat >> ~/.ssh/authorized_
keys"
```

With this, the public key on A should be appended to the *~/.ssh/authorized_keys* file on host B. Next, set up the *ssh-agent* daemon and use the *ssh-add* command to load the key. On host A, use the following commands:

```
$ eval `ssh-agent -s`
$ ssh-add
$ ssh-add -l
(should list your key fingerprint)
```

Now you are all set. If you log in on host A, you can talk to the remote machine without passwords; in place of this, your private key authenticates you to the other host. Instead of laboriously typing passwords, this lets you log in with the private key's password just once, and use the key for multiple remote logins. This is a bit complex, so please try it a few times to get it right.

SSH for simple port forwarding

There are basically two variants in port forwarding—local or remote port forwarding. In local port forwarding, you can set up a local port to be forwarded to some other machine. Whenever a connection is made to this port, the connection is forwarded over the secure channel, and a connection is made to the given host and port from the remote machine. In remote port forwarding, you can set up a remote port to be forwarded to a local machine when someone connects to the remote machine. This requires an SSH connection to be persistent between the hosts—a little bit of jugglery, since the port forwarding is session-based, and not permanent like on a firewall. In brief, the commands to set up local and remote port forwarding, respectively, are:

```
$ ssh -L 1234:remote:2345 remote
$ ssh -R 1234:remote:2345 remote
```

You also have to enable this in the */etc/ssh/sshd_config* on the remote host. You will often get a clear message to tell you

whether port forwarding succeeded or not.

The uses of this vary. In general, you can have a facility to log in to your local machine from any point on the Internet if you set up remote port forwarding to a public IP. This is why remote port forwarding seems more useful to me. In general, port forwarding is a vast topic requiring a lot of testing and thinking. It's not easy to set up; you have to know a lot of background information to get things working. In this article, we will also look at how to do this with a pf firewall rule.

SSH also allows you to set up a simple VPN between two networks. This requires the VPN option to be enabled in the *sshd_config* on both sides. A VPN tunnel helps you encrypt traffic and also access both networks seamlessly, but SSH VPNs are costly from a networking angle, since they add packet header overhead, and have the complexity of multiple TCP layers. You can use them in emergency situations when other VPN options are hard to set up, but IPsec VPNs are best for long-term set-ups. Or you could go for OpenVPN-type SSL VPNs without TCP.

SSH allows many different functions; the man pages and the commented configuration files tell you a lot. You could also turn off root logins in SSH in case you want to restrict root access from remote systems. But for the purpose of this article, I will stop right here, since I also have to cover other topics. Now let us move over to *netcat*, the Swiss army knife of networking.

Netcat

Netcat, also known as *nc*, is a very powerful tool for debugging issues in port forwarding, while setting up a simple TCP, UDP server or client; or even for testing UNIX domain sockets. It is also useful for port-scanning and sniffing, but let's look at only a few general uses of *netcat* here.

From a security point of view

The amazing *netcat* helps you figure out which ports are open, and which can be used to mount a security attack. *Netcat* does not always have to be run manually; it can be easily scripted, and you can do plenty of amazing things with it. *Netcat*'s simplicity is easily misunderstood, but its power lies in its simplicity. Let us look at some examples:

```
$ nc -l -p 1235
```

The previous simple command line sets up a TCP server at port 1235. You can then connect to it using the following command:

```
$ nc -v localhost 1235
```

Now you can chat between the two windows. In fact, *netcat* can help you write a simple chat program, or even figure out NAT traversal techniques used by programs like